

# 王道考研——组成原理

WWW.CSKAOYAN.COM

## 第二章 数据的表示和运算

### 考点3

# 定点数的表示和运算

### 考点3：定点数的表示与运算

|                            |                   |                   |         |                   |        |                  |                  |                    |
|----------------------------|-------------------|-------------------|---------|-------------------|--------|------------------|------------------|--------------------|
|                            | • 综合题43           | • 综合题43           | • 综合题44 | • 小题13<br>• 综合题44 | • 小题13 | • 小题13           |                  | • 综合题43<br>• 综合题44 |
| 2009                       | 2010              | 2011              | 2012    | 2013              | 2014   | 2015             | 2016             | 2017               |
| • 小题13<br>• 小题16<br>• 小题19 | • 小题13<br>• 综合题45 | • 小题13<br>• 综合题43 | • 小题13  | • 小题13            | • 小题13 | • 小题14<br>• 小题15 | • 小题12<br>• 小题14 |                    |
| 2018                       | 2019              | 2020              | 2021    | 2022              | 2023   | 2024             | 2025             |                    |

历年考频： 小题×15、综合题×8

# 定点数的表示与运算

## 【考点笔记】无符号数的表示

无符号数指整个机器字长的全部二进制位均为数值位，没有符号位，相当于数的绝对值。  
在 C 语言中对应 unsigned short、unsigned int 和 unsigned long 类型。

无符号数加法运算的结果通常保留低  $n$  位，舍弃高位。这样会产生两种结果：

① 保留的低  $n$  位数不能正确表达运算结果。此时，意味着运算结果超出了计算机所能表示的范围，有效数值进到了第  $n+1$  位，则称发生了“溢出”现象。

② 保留的低  $n$  位数能正确表达计算结果，即高位的舍去不影响运算结果。

短字长无符号数向长字长无符号数扩展时，只需在高位补 0 即可。

思考：无符号数减法怎么运算？

Eg: 4bit 无符号数 2-4    0010 - 0100

两个无符号数相减时，可能向更高位借位，导致PSW中的进位/借位标志位 CF=1

# 定点数的表示与运算

## 【考点笔记】有符号数的表示

有符号数的机器表示主要有原码、补码和反码，格式见表 2-2。

表 2-2 原码、补码和反码的机器表示

| 有符号数 | 符号位        | 数值部分（共 $n$ 位） |                                           |                                          | 0 的个数 |
|------|------------|---------------|-------------------------------------------|------------------------------------------|-------|
|      |            | 真值 $X > 0$    | 真值 $X < 0$                                |                                          |       |
|      |            |               | 整数                                        | 小数                                       |       |
| 原码   | 正 0<br>负 1 | $X$           | $2^n - X \quad (-2^n < X \leq 0)$         | $1 - X \quad (-1 < X \leq 0)$            | 2     |
| 补码   |            |               | $2^{n+1} + X \quad (-2^n \leq X < 0)$     | $2 + X \quad (-1 \leq X < 0)$            | 1     |
| 反码   |            |               | $2^{n+1} - 1 + X \quad (-2^n < X \leq 0)$ | $(2 - 2^{-n}) + X \quad (-1 < X \leq 0)$ | 2     |

原码、补码和反码之间的转换规律见表 2-3。

表 2-3 不同机器数之间的转换规律

|                                                  |                                                                                 |
|--------------------------------------------------|---------------------------------------------------------------------------------|
| $[X]_{\text{原}} \Leftrightarrow [X]_{\text{补}}$  | 对于正数：补码与原码的表示相同， $[X]_{\text{补}} = [X]_{\text{原}}$<br>对于负数：符号位不变，数值部分按位取反，末位加 1 |
| $[X]_{\text{补}} \Leftrightarrow [-X]_{\text{补}}$ | 连同符号位一起，各位取反，末位加 1                                                              |
| $[X]_{\text{原}} \Leftrightarrow [X]_{\text{反}}$  | 对于正数：反码与原码的表示相同， $[X]_{\text{反}} = [X]_{\text{原}}$<br>对于负数：符号位不变，数值部分按位取反       |

负数  $[X]_{\text{原}}$  转  $[X]_{\text{补}}$ ：从右往左找到第一个 1，除符号位之外，这个 1 左边的所有位取反

$[X]_{\text{补}}$  转  $[-X]_{\text{补}}$ ：从右往左找到第一个 1，这个 1 左边的所有位全部取反（常用于减法的预处理）

# 定点数的表示与运算

## 【考点笔记】C 语言中的整型数据类型

C 语言中的整型数据就是定点整数，一般用补码表示。

根据位数的不同，可以分为字符型(char)、整型(int)、短整型(short 或 short int)、长整型(long 或 long int)和无符号数(加上修饰符 unsigned)，见表 2-4。

表 2-4 C 语言中的整型数据类型及其范围

| 类型           | 所占位数 | 数的范围                   |
|--------------|------|------------------------|
| char         | 8    | -128~127               |
| short        | 16   | -32768~32767           |
| int          | 32   | -2147483648~2147483647 |
| unsigned int | 32   | 0~4294967295           |
| 32位编译器 long  | 32   | -2147483648~2147483647 |

32位编译器中定义 8B 整型可用 long long

64位编译器—— int 4B， long 8B

# 定点数的表示与运算

## 【考点笔记】定点整数的符号扩展

| 定点数 |       | 扩展方法                              |
|-----|-------|-----------------------------------|
| 正数  |       | 原符号位移动到新符号位上，新表示形式的所有附加位都用 0 进行填充 |
| 负数  | 原码    | 原符号位移动到新符号位上，新表示形式的所有附加位都用 0 进行填充 |
|     | 反码、补码 | 原符号位移动到新符号位上，新表示形式的所有附加位都用 1 进行填充 |

符号扩展的方法根据机器数的不同而不同。原码负数的符号扩展方法与正数相同，只不过此时符号位为 1 而已。补码负数的符号扩展方法：原有形式的符号位移动到新形式的符号位上，新表示形式的所有附加位都用 1 进行填充。反码负数的符号扩展方法：原有形式的符号位移动到新形式的符号位上，新表示形式的所有附加位都用 1 进行填充。

# 定点数的表示与运算

int→long, 短数据→长数据。多出来的那些位应该怎么填补?

Eg: 8位→16位

正整数（原、反、补码的表示都一样）

0,1011010 → 0,00000000 1011010

负整数:

原码: 1,1011010 → 1,00000000 1011010

反码: 1,0100101 → 1,11111111 0100101

补码: 1,0100110 → 1,11111111 0100110

正小数（原、反、补码的表示都一样）

0.1011010 → 0.1011010 00000000

负小数:

1.1011010 → 1.1011010 00000000

1.0100101 → 1.0100101 11111111

1.0100110 → 1.0100110 00000000

定点整数的符号扩展:

在原符号位和数值位中间添加新位, 正数都添0; 负数原码添0, 负数反、补码添1

定点小数的符号扩展:

在原符号位和数值位后面添加新位, 正数都添0; 负数原、补码添0, 负数反码添1

# 定点数的表示与运算

## 【考点笔记】有符号数和无符号数的转换

C 语言中的整型数据通常用补码表示，int 型变量的最高位是符号位，为 1 时表示负数，为 0 时表示正数。如果一个无符号数小于 32768 则最高位为 0，赋给一个 int 型变量后，得到与原值相同的正数。如果一个无符号数大于等于 32768 则最高位为 1，赋给一个 int 型变量后，就得到一个负整数。反之，当一个负整数赋给一个 unsigned 型变量时，得到的无符号数是一个大于 32768 的值。

因此，由有符号型数据转化为等长的 unsigned 型数据时，符号位解释为数据的一部分，也就是说，负数转化为无符号数时数值将发生变化。同理，由 unsigned 转化为有符号型数据时最高位解释为符号位，也可能发生数值的变化。

## 定点数的表示与运算

2016\_13. 有如下C语言程序段

```
short si = -32767;
```

```
unsigned short usi = si;
```

执行上述两条语句后，usi的值为\_\_\_\_\_。

A. -32767

B. 32767

C. 32768

D. 32769

答案：D

解析：

结合题干及选项可知，short为16位。因C语言中的数据在内存中为补码表示形式，si对应的补码二进制表示为：1000 0000 0000 0001B，最前面的一位“1”为符号位，表示负数，即-32767。由signed型转化为等长unsigned型数据时，符号位成为数据的一部分，也就是说，负数转化为无符号数（即正数），其数值将发生变化。Usi对应的补码二进制表示与si的表示相同，但表示正数，为32769。

# 定点数的表示与运算

## 【考点笔记】补码的加减运算

补码加减运算的特点如下 ( $n+1$  位):

| 运算        | 公式                                                             | 说明                                                                         |
|-----------|----------------------------------------------------------------|----------------------------------------------------------------------------|
| 补码定点数加法运算 | $[A+B]_{\text{补}} = [A]_{\text{补}} + [B]_{\text{补}} \pmod{M}$  | 符号位参与运算, 符号位产生的进位丢掉( $\pmod{M}$ )<br>定点小数时, 模 $M=2$ ; 定点整数时, 模 $M=2^{n+1}$ |
| 补码定点数减法运算 | $[A-B]_{\text{补}} = [A]_{\text{补}} + [-B]_{\text{补}} \pmod{M}$ |                                                                            |

## 【考点笔记】溢出的定义和判断

溢出是指运算结果超出了数的表示范围。仅当两个符号相同的数相加, 或两个符号相异的数相减才可能产生溢出。补码加/减运算的溢出判断方法见表 2-5。

表 2-5 补码定点数加/减运算溢出判断的方法

| 方法      | 溢出判断条件                             |
|---------|------------------------------------|
| 单符号位法   | 参加操作的两个数符号相同, 结果又与原操作数符号不同, 表示结果溢出 |
| 双符号位法   | 运算结果的两个符号位相同, 表示未溢出                |
|         | 运算结果的两个符号位不同, 表示溢出, 最高位符号位代表真正的符号  |
| 符号位和进位法 | 若符号位的进位与最高数位的进位相同, 表示没有溢出, 否则表示溢出  |

## 扩展（了解一哈）

### ➤ 条件码:

**OF** (Overflow Flag) 溢出标志。溢出时为1, 否则置0。

**SF** (Sign Flag) 符号标志。结果为负时置1, 否则置0。

**ZF** (Zero Flag) 零标志, 运算结果为0时ZF位置1, 否则置0。

**CF** (Carry Flag) 进位/借位标志, 进位/借位时置1, 否则置0。

**AF** (Auxiliary carry Flag) 辅助进位标志, 记录运算时第3位（半个字节）产生的进位置。有进位时1, 否则置0。

**PF** (Parity Flag) 奇偶标志。结果操作数中1的个数为偶数时置1, 否则置0。

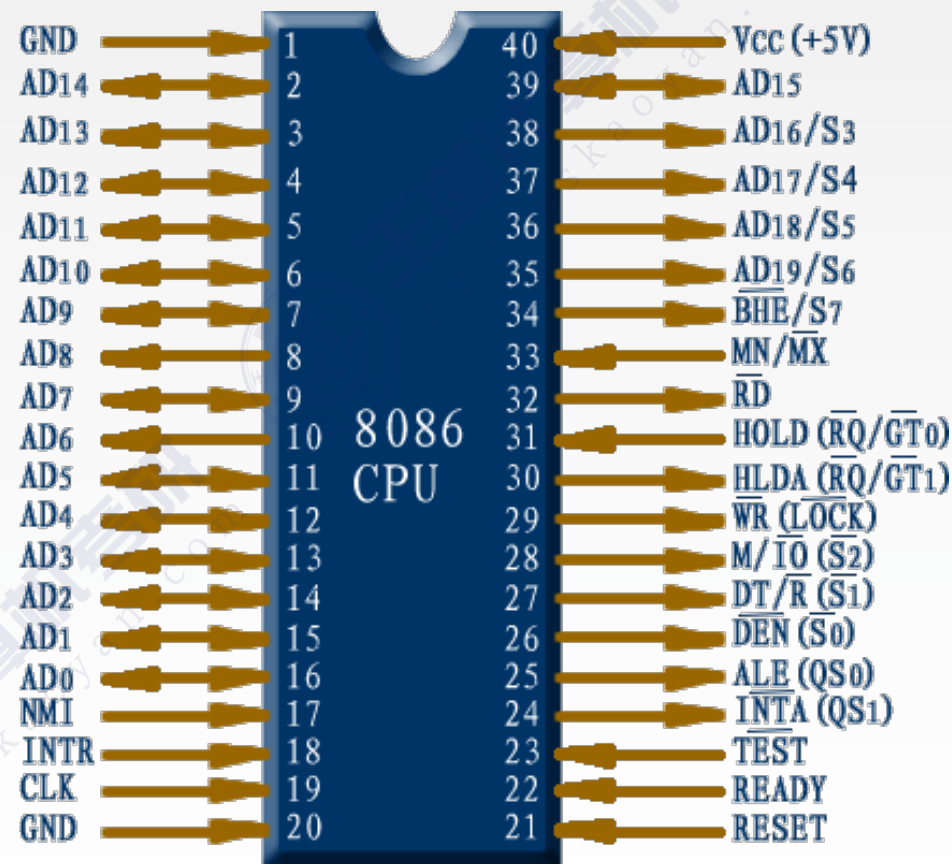
### ➤ 控制标志位:

**DF** (Direction Flag) 方向标志, 在串处理指令中控制信息的方向。

**IF** (Interrupt Flag) 中断标志。

**TF** (Trap Flag) 陷阱标志。

|    |    |    |    |    |    |    |    |    |    |   |    |   |    |   |    |
|----|----|----|----|----|----|----|----|----|----|---|----|---|----|---|----|
| 15 | 14 | 13 | 12 | 11 | 10 | 9  | 8  | 7  | 6  | 5 | 4  | 3 | 2  | 1 | 0  |
|    |    |    |    | OF | DF | IF | TF | SF | ZF |   | AF |   | PF |   | CF |



- NMI:不可屏蔽中断请求信号。常用于处理电源掉电紧急情况。
- INTR:可屏蔽中断请求信号。

## 溢出判断



2014\_13. 若 $x=103$ ,  $y=-25$ , 则下列表达式采用8位定点补码运算实现时, 会发生溢出的是\_\_\_\_\_。

A.  $x+y$

B.  $-x+y$

C.  $x-y$

D.  $-x-y$

答案: C

解析:

8位定点补码表示的数据范围为 $-128 \sim 127$ , 若运算结果超出这个范围则会溢出, A选项 $x+y=103-25=78$ , 符合范围, A排除; B选项 $-x+y=-103-25=-128$ , 符合范围, B排除; D选项 $-x-y=-103+25=-78$ , 符合范围, D排除; C选项 $x-y=103+25=128$ , 超过了127, 选C。  
该题也可按照二进制写出两个数进行运算, 观察运算的进位信息得到结果, 不过这种方法更为麻烦和耗时, 在实际考试中并不推荐。

## 溢出判断

2010\_13. 假定有 4 个整数用 8 位补码分别表示  $r1=FEH$ ,  $r2=F2H$ ,  $r3=90H$ ,  $r4=F8H$ , 若将运算结果存放在一个 8 位寄存器中, 则下列运算中会发生溢出的是\_\_\_\_\_。

A.  $r1 \times r2$

B.  $r2 \times r3$

C.  $r1 \times r4$

D.  $r2 \times r4$

# 定点数的表示与运算

算数移位：（符号位不参与移动）

|    | 码 制      | 添 补 代 码 |
|----|----------|---------|
| 正数 | 原码、补码、反码 | 0       |
| 负数 | 原码       | 0       |
|    | 补码       | 左移添 0   |
|    |          | 右移添 1   |
|    | 反码       | 1       |

逻辑移位：所有位参与移动，无论左移、右移，都是添0

## 定点数的表示与运算

2018\_16. 整数x的机器数为1101 1000，分别对x进行逻辑右移1位和算术右移1位操作，得到的机器数各是\_\_\_\_\_。

A. 1110 1100、1110 1100

C. 1110 1100、0110 1100

B. 0110 1100、1110 1100

D. 0110 1100、0110 1100

答案：B

解析：

逻辑移位：左移和右移空位都是补0，并且所有数字参与移动；而算术移位：符号位不参与移动，右移空位补符号位，左移空位补0。根据该规则，轻松选出B选项。

## 考点4

# C语言中各种 数据的转换

## 考点4: C语言中各种数据的转换

|      |            |             |            |      |      |      |            |             |
|------|------------|-------------|------------|------|------|------|------------|-------------|
|      | • 小题<br>14 | • 综合<br>题43 | • 小题<br>13 |      |      |      | • 小题<br>13 | • 综合<br>题43 |
| 2009 | 2010       | 2011        | 2012       | 2013 | 2014 | 2015 | 2016       | 2017        |
|      |            |             |            |      |      |      | • 小题<br>12 |             |
| 2018 | 2019       | 2020        | 2021       | 2022 | 2023 | 2024 | 2025       |             |

历年考频: 小题×4、综合题×2

# 各种精度数据的转换

## 【考点笔记】不同类型数据的转换

双目运算符两侧的操作数类型要求必须一致，当出现两侧操作数类型不同时，系统会将较低类型转换为较高类型。主要有以下三种基本形式。

| 数据类型的转换            | 转换原则                          |
|--------------------|-------------------------------|
| 同一类型但字长不同的数据间的转换   | 由字长短的转为字长长的                   |
| 定点方式与浮点方式间的转换      | 由定点数转换为浮点数                    |
| 整型数中的有符号数和无符号数间的转换 | 机器数不变，只是看待符号位的角度不同，代表的数值也随之不同 |

## 【考点笔记】数据运算中的类型转换

不同类型数据的混合运算时，遵循的原则是“类型提升”，即较低类型转换为较高类型。类型转换的常用规律为 `char→int→long→float→double`。

如一个 `long` 型数据与一个 `int` 型数据一起运算，需先将 `int` 型转换为 `long` 型，然后再进行运算，结果为 `long` 型。如果 `float` 型和 `double` 型数据一起运算，虽然它们同为实型，但两者精度不同，仍要先将 `float` 型转换成 `double` 型再进行运算，结果亦为 `double` 型。所有这些转换都是由系统自动进行的，这种转换称为隐式类型转换。

## 各种精度数据的转换

C 语言中的强制类型转换格式为“`TYPE b = (TYPE) a`”，强制类型转换后，返回一个具有 `TYPE` 类型的数值，这种强制类型转换操作并不会改变操作数本身。

在类型转换中，从 `int` 转换为 `float` 时，虽然不会发生溢出，但由于尾数位数的关系，可能有数据舍入，而转换为 `double` 则能保留精度。从 `float` 或 `double` 转换为 `int` 时，小数部分被截断，且由于 `int` 的表示范围更小，还可能发生溢出。`Double` 转换为 `float` 亦是如此。

数据转换时应注意：

① 数据的截取与保留。当一个浮点数转换为整数时，浮点数的小数部分全部舍去，并按整数形式存储。因此整数部分不能超过整型数允许的最大范围，否则溢出。

② 精度的丢失。四舍五入会丢失一些精度，截去小数也会丢失一些精度。数据由 `long` 型转换为 `float` 或 `double` 型时，有可能在存储时不能准确地表示该长整数的有效数字，精度也会受到影响。

③ 转换结果的不确定性。当较长的整数转换为较短的整数时，要将高位截去，如 `long` 型转换为 `short` 型，只保留低 16 位，这样就会产生很大的误差。浮点数降格时，如 `double` 型转换为 `float` 型，当数值超过了 `float` 型的表示范围时，所得到的结果也是不确定的。

## 各种精度数据的转换

2010\_14. 假定变量i、f和d的数据类型分别为int、float和double (int用补码表示, float和double分别用IEEE754单精度和双精度浮点数格式表示), 已知 $i=785$ ,  $f=1.5678e3$ ,  $d=1.5e100$ 。若在32位机器中执行下列关系表达式, 则结果为“真”的是\_\_\_\_\_。

I.  $i==(int)(float)i$

II.  $f==(float)(int)f$

III.  $f==(float)(double)f$

IV.  $(d+f)-d==f$

A. 仅I和II

B. 仅I和III

C. 仅II和III

D. 仅III和IV

答案: B

解析:

题中三种数据类型的精度从低到高为 $int \rightarrow float \rightarrow double$ , 从低到高的转换通常可以保持其值不变, I和III正确, 而从高到低的转换可能会有数据的舍入, 从而损失精度。对于II, 先将float型的f转换为int型, 小数点后的数位丢失, 故其结果不为真。对于IV, 初看似乎没有问题, 但浮点运算 $d+f$ 时需要对其对阶, 对阶后f的尾数有效位被舍去而变为0, 故 $d+f$ 仍然为d, 再减去d后结果为0, 故IV结果不为真。

此外, 根据不同类型数据混合运算的“类型提升”原则, 在IV中, 等号左端的类型为double型, 结果不为真。

考点5（上）

# IEEE 754 标准

## 考点5（上）：IEEE 754标准

|        |      |        |        |        |        |        |        |             |
|--------|------|--------|--------|--------|--------|--------|--------|-------------|
|        |      | • 小题13 | • 小题14 | • 小题13 | • 小题14 |        |        | • 综合题<br>43 |
| 2009   | 2010 | 2011   | 2012   | 2013   | 2014   | 2015   | 2016   | 2017        |
| • 小题14 |      | • 小题13 | • 小题14 | • 小题14 | • 小题14 | • 小题14 | • 小题14 | • 小题13      |
| 2018   | 2019 | 2020   | 2021   | 2022   | 2023   | 2024   | 2025   |             |

历年考频： 小题×11、 综合题×1

### 【考点笔记】移码的定义

移码是在真值  $X$  的基础上加一个偏置值，相当于  $X$  在数轴上向正方向偏移了若干单位，这就是“移码”名称的由来，其定义为

$$[X]_{\text{移}} = \text{偏置值} + X$$

对于字长为  $n+1$  的定点整数而言，偏置值通常取  $2^n$ 。

移码可视为无符号数，码值的大小直观地反映了真值的大小，采用移码表示浮点数的阶码，使得浮点运算中比较阶码的大小变得很方便。

**【考点笔记】IEEE 754 标准的浮点数**

IEEE 754 标准规定常用的浮点数格式有短浮点数（单精度、float 型）、长浮点数（双精度、double 型）和临时浮点数，见表 2-7。

**表 2-7 IEEE 754 浮点数的格式**

| 类型           | 数符 | 阶码 | 尾数数值 | 总位数 | 偏置值   | 真值                                    |
|--------------|----|----|------|-----|-------|---------------------------------------|
| 短浮点数（float）  | 1  | 8  | 23   | 32  | 127   | $(-1)^S \times 1.m \times 2^{E-127}$  |
| 长浮点数（double） | 1  | 11 | 52   | 64  | 1023  | $(-1)^S \times 1.m \times 2^{E-1023}$ |
| 临时浮点数        | 1  | 15 | 64   | 80  | 16383 |                                       |

注：S 表示数符（0 正 1 负），m 表示尾数（原码表示），E 表示阶码（移码表示）。

注意：

- ①尾数表示小数部分，小数点之前隐含一个“1”
- ②float 型的阶码用移码表示，偏置值是-127，而不是-128
- ③阶码全0表示无穷小、全1表示无穷大，因此float 型的阶码的真值范围是 -126~127

## IEEE 754标准

2018\_14. IEEE 754单精度浮点格式表示的数中，最小的规格化正数是\_\_\_\_\_。

A.  $1.0 \times 2^{-126}$

B.  $1.0 \times 2^{-127}$

C.  $1.0 \times 2^{-128}$

D.  $1.0 \times 2^{-149}$

答案：A

解析：

IEEE754单精度浮点数的符号位、阶码位、尾数位（省去正数位1）所占的位数分别是1、8、23位。最小正数，数符位取0，移码的取值范围是1-254，取1，得阶码值 $1-127=-126$ （127为我们规定的偏置值），尾数取全0，最终推出最小规格化正数为A选项。

表 2-8 IEEE 754 浮点数的范围

| 格式  | 最小值                                                   | 最大值                                                                                                    |
|-----|-------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| 单精度 | $E = 1, m = 0$<br>$1.0 \times 2^{1-127} = 2^{-126}$   | $E = 254, m = .111 \cdots 1,$<br>$1.111 \cdots 1 \times 2^{254-127} = 2^{127} \times (2-2^{-23})$      |
| 双精度 | $E = 1, m = 0$<br>$1.0 \times 2^{1-1023} = 2^{-1022}$ | $E = 2046, m = .1111 \cdots 1,$<br>$1.111 \cdots 1 \times 2^{2046-1023} = 2^{1023} \times (2-2^{-52})$ |

## IEEE 754标准

2014\_float型数据常用IEEE754单精度浮点格式表示。假设两个float型变量x和y分别存放在32位寄存器f1和f2中，若(f1)=CC90 0000H，(f2)=B0C0 0000H，则x和y之间的关系为（ ）。

- A.  $x < y$ 且符号相同
- B.  $x < y$ 且符号不同
- C.  $x > y$ 且符号相同
- D.  $x > y$ 且符号不同

答案：A

解析：

(f1)和(f2)对应的二进制分别是 $(110011001001.....)_2$ 和 $(101100001100.....)_2$ ，根据IEEE754浮点数标准，可知(f1)的数符为1，阶码为10011001，尾数为1.001，而(f2)的数符为1，阶码为01100001，尾数为1.1，则可知两数均为负数，符号相同，B、D排除，(f1)的绝对值为 $1.001 \times 2^{26}$ ，(f2)的绝对值为 $1.1 \times 2^{-30}$ ，则(f1)的绝对值比(f2)的绝对值大，而符号为负，真值大小相反，即(f1)的真值比(f2)的真值小，即 $x < y$ ，选A。

此题还有更为简便的算法，(f1)与(f2)的前4位为1100与1011，可以看出两数均为负数，而阶码用移码表示，两数的阶码头三位分别为100和011，可知(f1)的阶码大于(f2)的阶码，又因为是IEEE754规格化的数，尾数部分均为1.xxx，则阶码大的数，真值的绝对值必然大，可知(f1)真值的绝对值大于(f2)真值的绝对值，因为都为负数，则 $(f1) < (f2)$ ，即 $x < y$ 。

考点5（下）

# 浮点数运算

## 考点5（下）：浮点数运算

|        |        |      |      |      |      |        |      |                    |
|--------|--------|------|------|------|------|--------|------|--------------------|
| • 小题13 | • 小题14 |      |      |      |      | • 小题14 |      | • 综合题43<br>• 综合题44 |
| 2009   | 2010   | 2011 | 2012 | 2013 | 2014 | 2015   | 2016 | 2017               |
|        |        |      |      |      |      |        |      |                    |
| 2018   | 2019   | 2020 | 2021 | 2022 | 2023 | 2024   | 2025 |                    |

历年考频： 小题×3、综合题×2

# 浮点数运算

## 【考点笔记】浮点数的加减运算

浮点数运算的特点是阶码运算和尾数运算分开进行（一律采用补码），难点在于对阶、结果规格化、溢出判断这三步。运算过程如下所示。

### (1) 对阶

小阶向大阶看齐。将阶码小的尾数右移一位，阶加 1，直到两个数的阶码相等为止。

### (2) 尾数求和

将对阶后的尾数按定点数加（减）运算规则运算。

### (3) 规格化

补码规格化形式（归纳：尾数的符号位和最高有效数位不同）如下所示。

| 尾数大于 0                                      | 尾数小于 0                                      |
|---------------------------------------------|---------------------------------------------|
| $[S]_{\text{补}} = 00.1 \times \dots \times$ | $[S]_{\text{补}} = 11.0 \times \dots \times$ |

规格化分为左规和右规两种：

① 左规：当尾数出现  $00.0 \times \dots$  或  $11.1 \times \dots$  时，需左规，即尾数左移 1 位，和的阶码减 1。

② 右规：当尾数求和结果溢出（如尾数为  $10. \times \dots$  或  $01. \times \dots$ ）时，需右规，即尾数右移一位，和的阶码加 1。右规最多一次。

# 浮点数运算

## (4) 舍入（在对阶和右规的过程中）

| 简单丢弃法    | 误差最大                                 |
|----------|--------------------------------------|
| 0 舍 1 入法 | 被移去的最高数值位为 0，则舍去；被移去的最高数值位为 1，则末位加 1 |
| 恒置 1 法   | 不论丢掉的最高数值位是 1 还是 0，都使尾数末位恒置 1        |

## (5) 溢出判断（双符号位补码）

对尾数求和结果进行规格化后：

① 若为左规，不会发生溢出。

② 若为右规，溢出与否是由阶码的符号决定的，阶码的符号位不同表示溢出。当阶码的符号位出现“01”时，即阶码大于最大阶码，表示上溢，进入中断处理；当阶码的符号位出现“10”时，即阶码小于最小阶码，表示下溢，按机器零处理。

## 浮点数的运算

2009\_13. 浮点数加、减运算过程一般包括对阶、尾数运算、规格化、舍入和判溢出等步骤。设浮点数的阶码和尾数均采用补码表示，且位数分别为5位和7位（均含2位符号位）。若有两个数 $X=2^7 \times 29/32$ ， $Y=2^5 \times 5/8$ ，则用浮点加法计算 $X+Y$ 的最终结果是\_\_\_\_\_。

A. 00111 1100010  
C. 01000 0010001

B. 00111 0100010  
D. 发生溢出

答案：D

解析：

X的浮点数格式为00,111;00,11101（分号前为阶码，分号后为尾数），Y的浮点数格式为00,101;00,10100。然后根据浮点数的加法步骤进行运算。

第一步：对阶。X、Y阶码相减，即 $00,111 - 00,101 = 00,111 + 11,0111 = 00,010$ ，可知X的阶码比Y的阶码大2（这一步可直接目测）。根据小阶向大阶看齐的原则，将Y的阶码加2，尾数右移2位，将Y变为00,111; 00,00101。

第二步：尾数相加。即 $00,11101 + 00,00101 = 01,00010$ ，尾数相加结果符号位为01，故需右规。

第三步：规格化。将尾数右移1位，阶码加1，得 $X+Y$ 为01,000; 00,10001。

第四步：判溢出。阶码符号位为01，说明发生溢出。

本题容易误选选项B、C，这是因为选项B、C本身并没有计算错误，只是它们不是最终结果，选项B少了第3步，选项C少了第4步。

## 考点 6

# 数据的对齐和 大小端存储

## 考点 6: 数据的对齐和大小端存储

|            |            |            |      |      |      |            |            |      |
|------------|------------|------------|------|------|------|------------|------------|------|
|            |            |            |      |      |      |            | • 小题<br>14 |      |
| 2009       | 2010       | 2011       | 2012 | 2013 | 2014 | 2015       | 2016       | 2017 |
| • 小题<br>15 | • 小题<br>15 | • 小题<br>14 |      |      |      | • 大题<br>44 | • 小题<br>15 |      |
| 2018       | 2019       | 2020       | 2021 | 2022 | 2023 | 2024       | 2025       |      |

历年考频: 小题×5、综合题×1

# 数据存储的小端方式与大端方式

## 【考点笔记】数据存储的小端方式与大端方式

现代计算机多采用字节编址方式。对于占用多个存储单元的某些类型数据，如 int、float、double 等，有两种不同的存储方式：大端方式和小端方式。

- 小端方式：先存储低位字节，后存储高位字节。
- 大端方式：先存储高位字节，后存储低位字节。

例如，将一个 32 位的整数 0x12345678 存放到一个 int 变量中，这个 int 变量采用大端或小端方式在内存中的存储见表 2-1。为了简单起见，这里使用 OP0 表示一个 32 位数据的最高字节，使用 OP3 表示一个 32 位数据最低字节。

表 2-1 采用小端或大端方式在内存中的存储

| 地址偏移 | 0×00     | 0×01     | 0×02     | 0×03     |
|------|----------|----------|----------|----------|
| 小端方式 | 78 (OP3) | 56 (OP2) | 34 (OP1) | 12 (OP0) |
| 大端方式 | 12 (OP0) | 34 (OP1) | 56 (OP2) | 78 (OP3) |

小端方式和大端方式的区别是字中的字节的存储顺序不同，而字的存储顺序是相同的。采用大端方式进行数据存放符合人类的正常思维。

## 数据的存储方式-大端与小端

2016\_14. 某计算机字长为32位，按字节编址，采用小端（Little Endian）方式存放数据。假定有一个double型变量，其机器数表示为1122 3344 5566 7788H，存放在0000 8040H开始的连续存储单元中，则存储单元0000 8046H中存放的是\_\_\_\_\_。

- A. 22H                      B. 33H                      C. 77H                      D. 66H

答案：A

解析：

大端方式：一个字中的高位字节（Byte）存放在内存中这个字区域的低地址处。小端方式：一个字中的低位字节（Byte）存放在内存中这个字区域的低地址处。依此分析，各字节的存储分配如下表所示：

| 地址 | 0000 8040H | 0000 8041H | 0000 8042H | 0000 8043H |
|----|------------|------------|------------|------------|
| 内容 | 88H        | 77H        | 66H        | 55H        |
| 地址 | 0000 8044H | 0000 8045H | 0000 8046H | 0000 8047H |
| 内容 | 44H        | 33H        | 22H        | 11H        |

从而存储单元0000 8046H中存放的是：22H。

# 边界对齐方式

## 【考点笔记】边界对齐方式

假设存储字长为 32 位，可按字节、半字、字寻址。在对准边界的 32 位字长的计算机中，半字地址是 2 的整数倍，字地址是 4 的整数倍，当所存数据不能满足此要求时，可填充一个或多个空白字节。这样无论所存的数据是字节、半字还是字，均可一次访存取出。

当数据不按边界对齐方式存储时，一个字的数据可能在两个存储字中，此时需要两次访存，并对高低字节的位置进行调整后才能取得一个字，影响了系统的效率，如图 2-1 所示。

| 存储器 |    |    |    | 存储器 |    |    |    | 字地址 |
|-----|----|----|----|-----|----|----|----|-----|
| 3   | 2  | 1  | 0  | 3   | 2  | 1  | 0  | 0   |
| 7   | 6  | 5  | 4  | 7   | 6  | 5  | 4  | 4   |
| 11  | 10 | 9  | 8  | 11  | 10 | 9  | 8  | 8   |
| 15  | 14 | 13 | 12 | 15  | 14 | 13 | 12 | 12  |

(a) 不按边界对齐

(b) 按边界对齐

图 2-1 两种数据的存放方式

## 边界对齐方式

### 【考点笔记】C 语言中 struct 存储的边界对齐问题

在 C 语言 struct 中，存储对齐有几个重要条件：①每个成员按其类型的方式对齐，char 的对齐值为 1，short 为 2，int 为 4，单位字节。②struct 长度必须是成员中最大的对齐值的整数倍（不够就补空字节），以便在处理数组时保证每一项都边界对齐。

我们先看两个例子（32bit，x86 环境，gcc 编译器）：

```
struct A{  
    int a;  
    char b;  
    short c;  
}
```

```
struct B{  
    char b;  
    int a;  
    short c;  
}
```



WHY?

结果却是：sizeof(A) = 8，sizeof(B) = 12。

|        |   |         |
|--------|---|---------|
| int a  |   |         |
| char b | 空 | short c |

|         |   |   |   |
|---------|---|---|---|
| char b  | 空 | 空 | 空 |
| int a   |   |   |   |
| short c |   | 空 | 空 |

# 数据的存储方式-边界对齐

2012\_15. 某计算机存储器按字节编址，采用小端方式存放数据。假定编译器规定int型和short型长度分别为32位和16位，并且数据按边界对齐存储。某C语言程序段如下：

```
struct{
    int    a;
    char   b;
    short  c;
} record;
record.a=273;
```

|    |                    |                    |                    |                    |
|----|--------------------|--------------------|--------------------|--------------------|
| 地址 | 0xC008             | 0xC009             | 0xC00A             | 0xC00B             |
| 内容 | record.a<br>(0x11) | record.a<br>(0x01) | record.a<br>(0x00) | record.a<br>(0x00) |
| 地址 | 0xC00C             | 0xC00D             | 0xC00E             | 0xC00F             |
| 内容 | record.b           | -                  | record.c           | record.c           |

若record变量的首地址为0xC008，则地址0xC008中内容及record.c的地址分别为\_\_\_\_\_。

- A. 0x00、0xC00D
- B. 0x00、0xC00E
- C. 0x11、0xC00D
- D. 0x11、0xC00E

答案：D

解析：

尽管record大小为7个字节（成员a有4个字节，成员b有1个字节，成员c有2个字节），由于数据按边界对齐方式存储（见考点笔记），故record共占用8个字节。record.a的十六进制表示为0x00000111，由于采用小端方式存放数据，故地址0xC008中内容应为低字节0x11；record.b只占1个字节，后面的一个字节留空；record.c占2个字节，故其地址为0xC00E。各字节的存储分配如图所示。